Observation Kalman Filtering

Denoising observed smiles without confusing noise with gaps

Vol-Fitter Technical Notes, No. 15

Vol-Fitter

July 3, 2026

Abstract

Prior persistence and Kalman filtering both pull a smile away from a raw, current-market fit, so they are easy to confuse in implementation. They should not be confused mathematically. Prior persistence is a *gap regularizer*: it keeps unobserved or weakly identified smile functionals near a transported prior and turns itself off where today's quotes already identify the functional. A Kalman observation filter is a *temporal state estimator*: it assumes the current smile is observed through noisy, sometimes contradictory quotes, predicts the smile from the previous filtered state, and updates according to the relative precision of the prediction and the observation. This note proposes a clean Vol-Fitter architecture for adding such a filter. The state is not raw option mids, but compact arbitrage-safe smile handles; the transition is the existing SSR transport; the measurement is a data-only fit with a covariance derived from bid–ask width, quote density, fit residuals and local inconsistency; and active mode is a one-stage MAP calibration so the same quotes are not counted twice. A small case file shows the key distinction: dense but contradictory close strikes should inflate observation noise and be denoised by the Kalman layer, while genuinely unquoted wings should be governed by prior persistence.

Contents

1 The production symptom

A sparse wing and a noisy cluster are different failures.

In a sparse wing, the desk has almost no market information. A data-only fit can extend the ATM skew into the tail and create a new signal where no option actually traded. Note 13 solved that problem with an activation gate: the prior is strong in the gap and exactly off where the data is sufficient.

In a noisy cluster, the desk has information, but it is internally inconsistent. Two adjacent strikes may imply a local kink because one market is stale, a wide bid–ask band may make the mid meaningless, or an American wing repair may have preserved spread while lowering the reliable rank of the wing. The right answer is not "fill with yesterday"; the right answer is "infer the latent current smile under observation noise."

Invariant

1. Prior persistence acts only on functionals not identified by the current market at the required precision.
2. Observation filtering acts only through an explicit prediction covariance and an explicit measurement covariance.
3. The same quote cannot be used once to create a filtered posterior and then again as an independent calibration datum.
4. The filter state is an arbitrage-safe coordinate, such as ATM handles or operator values, never a free raw-IV curve.
5. Every filtered output reports prediction, observation, innovation, gain and posterior uncertainty.

The two features therefore answer two different questions:

Prior persistence:	Where do we lack enough observations to identify the smile?
Kalman filtering:	Given observations here, how noisy are they?

The implementation should preserve that distinction all the way to the UI.

2 Two regularizers, two measures

Let $x_t \in \mathbb{R}^d$ denote the latent smile state of one node at snapshot time t: for the first implementation this should be a compact vector of handles, for example

$$x_t = (\sigma_{\mathrm{atm},t}, s_{\mathrm{atm},t}, \kappa_{\mathrm{atm},t})^{\mathsf{T}},$$

possibly extended later by signed operators such as RR25, BF25 and var-swap. Let q_t be the prepared quote set after forwards, de-Americanization, event-clock conversion, bid–ask band construction and quote edits.

2.1 Prior persistence

Prior persistence is a penalty in a calibration objective. With a prior state x_t^0 transported to the current forward, and functionals $O_j : \mathbb{R}^d \to \mathbb{R}$, Note 13 has the universal form

$$\mathcal{L}_{\text{persist}}(x) = \tfrac{1}{2} \sum_j \lambda_j^{\text{gap}} \left(\frac{O_j(x) - O_j(x_t^0)}{a_j} \right)^2, \qquad \lambda_j^{\text{gap}} = \lambda_j^0 \max\left(1 - \frac{\pi_j^{\text{obs}}}{\pi_j^{\text{req}}}, 0\right)^\gamma. \tag{1}$$

The gate is driven by *identifiability*: when current quotes identify functional O_j at or above the required precision, $\lambda_j^{\text{gap}} = 0$. Thus prior persistence is not primarily a denoiser. It is a rule for refusing to invent signal outside the support of the observation.

2.2 Observation filtering

A Kalman filter is instead a sequential estimator. It starts from the previous posterior law for x_{t-1}, transports it forward, then combines the transported prediction with today's noisy observation. In the linear-Gaussian handle model,

$$x_t = \Phi_t x_{t-1} + u_t + \eta_t, \qquad \eta_t \sim \mathcal{N}(0, Q_t), \tag{2}$$

$$z_t = H_t x_t + \epsilon_t, \qquad \epsilon_t \sim \mathcal{N}(0, R_t), \tag{3}$$

where z_t is not a raw quote vector; it is a noisy handle estimate extracted from today's prepared quotes. Q_t is process covariance: time elapsed, spot move, event risk and transport distance. R_t is measurement covariance: bid–ask width, quote density, stale fields, de-Americanization repairs, band slack and contradictory close strikes.

> **Heuristic**
>
> Persistence says "do not put signal where the market did not speak." Filtering says "when the market speaks with a noisy voice, infer what it probably meant." Both can pull toward a previous surface, but the sufficient statistic is different: persistence uses a *gap*; filtering uses a *covariance*.

3 The Kalman object

The Vol-Fitter should implement the filter in covariance form, matching the graph posterior style of Note 14. For a direct handle observation $H_t = I$, the form is especially transparent, but the equations below allow a subset or a linear operator measurement.

Definition 1 (Predicted and observed handle laws)**.** At snapshot t, the prediction law and the measurement law are

$$x_t \,|\, \mathcal{F}_{t-1} \sim \mathcal{N}(m_t^-, P_t^-), \qquad z_t \,|\, x_t \sim \mathcal{N}(H_t x_t, R_t).$$

m_t^- is the transported previous filtered state; P_t^- is its transported uncertainty plus process noise. z_t and R_t are extracted from today's prepared quotes without using prior persistence.

The central update is

$$\boxed{\begin{aligned} S_t &= H_t P_t^- H_t^\mathsf{T} + R_t, \\ K_t &= P_t^- H_t^\mathsf{T} S_t^{-1}, \\ m_t^+ &= m_t^- + K_t(z_t - H_t m_t^-), \\ P_t^+ &= (I - K_t H_t) P_t^- (I - K_t H_t)^\mathsf{T} + K_t R_t K_t^\mathsf{T}. \end{aligned}} \tag{4}$$

The last line is the Joseph form; it is a little longer than $P_t^+ = (I - K_t H_t) P_t^-$, but it preserves symmetry and positive semidefiniteness better in floating point.

Proposition 1 (Precision-weighted shrinkage)**.** *In one dimension with $H_t = 1$, prediction variance p and measurement variance r, the posterior mean is*

$$m^+ = \frac{r}{p+r} m^- + \frac{p}{p+r} z,$$

and the Kalman gain is $K = p/(p+r)$.

Proof. The innovation variance is $S = p + r$, so $K = p/S$. Substituting in $m^+ = m^- + K(z - m^-)$ gives $m^+ = (1 - K)m^- + Kz = r(p+r)^{-1} m^- + p(p+r)^{-1} z$. □

Proposition 1 is the whole desk intuition. If today's observation is tight ($r \ll p$), the filtered smile moves to the market. If today's observation is noisy ($r \gg p$), it stays near the transported prediction. The movement is not controlled by whether the region is observed; it is controlled by the relative uncertainties of prediction and observation.

4 Where the observation covariance comes from

The hard part is not the Kalman algebra. The hard part is building a measurement covariance R_t that reflects the quote set honestly.

4.1 Do not filter raw mids

Raw contracts are a poor state space. The OTM side changes when the forward moves; contracts appear and disappear; American contracts need early-exercise removal; a strike can be inside a wide spread but far from its midpoint; and a raw implied-vol vector is not guaranteed to remain butterfly-free after smoothing. The filter should begin after the existing quote-prep pipeline in `backend/volfit/api/quotes.py`.

4.2 Extract a noisy handle observation

The recommended v1 is a two-step observation extractor:

$$z_t = \arg\min_x \; \mathcal{L}_{\text{quote}}(x; q_t) + \mathcal{L}_{\text{intrinsic}}(x),$$

where $\mathcal{L}_{\text{quote}}$ is the same vega-normalized/bid–ask-aware loss as Note 07, and $\mathcal{L}_{\text{intrinsic}}$ contains only intrinsic model regularization needed to obtain a valid no-arbitrage smile. It must not

include temporal prior persistence. The extractor then maps the fitted smile to handles $z_t = (\hat{\sigma}_{\text{atm}}, \hat{s}_{\text{atm}}, \hat{\kappa}_{\text{atm}})$.

Near the optimum, linearize the quote residuals as

$$r(\theta) \approx r(\hat{\theta}) + J(\theta - \hat{\theta}).$$

If the handle map is $x = g(\theta)$ with Jacobian $G = \partial g / \partial \theta$, then an observed-information approximation is

$$\mathcal{I}_\theta = J^\mathsf{T} W J + \Lambda_{\text{intrinsic}}, \qquad R_x \approx G \mathcal{I}_\theta^\dagger G^\mathsf{T}. \tag{5}$$

Equivalently, if the model is already parameterized by handles, use the handle Hessian directly. In either case the covariance is inflated by realized inconsistency:

$$R_t = \rho_t R_x, \qquad \rho_t = \max\left(1, \frac{\chi_t^2}{\max(m - d, 1)}\right), \qquad \chi_t^2 = r(\hat{\theta})^\mathsf{T} W r(\hat{\theta}). \tag{6}$$

This is the mechanism that makes close-strike contradictions visible to the filter. A contradictory cluster may have many quotes, but if it cannot be fitted within its stated noise, ρ_t grows and the Kalman gain falls in the directions the cluster does not reliably identify.

Remark 1 (Bands and covariance)**.** For bid–ask mode, quotes inside the band should not pretend to identify the mid. In a practical implementation, inactive hinge rows contribute little or nothing to $J^\mathsf{T} W J$, while the weak mid anchor contributes only at its small anchor weight. This matches Note 07's semantics: inside the spread the market is a set, not a point.

4.3 A cheaper v1 precision builder

The full covariance (5) is the right target, but a first implementation can reuse the existing precision vocabulary:

$$R_t^{-1} = \text{diag}(c_h)\, \pi_{\text{fit}}\, f_{\text{density}}\, f_{\text{spread}}\, f_{\text{fresh}}\, f_{\text{repair}}\, f_{\text{resid}},$$

where c_h is per-handle confidence, π_{fit} is inverse squared RMS with a floor and cap, and the scalar factors are exactly the kind already used by `backend/volfit/calib/precision.py` and `backend/volfit/graph/precision.py`. This is less elegant than a full Jacobian covariance but enough for an overlay pilot.

5 The prediction law

The prediction must transport the previous filtered smile before comparing it with today's quotes. For a handle state,

$$m_t^- = \mathcal{T}_t(m_{t-1}^+), \qquad P_t^- = A_t P_{t-1}^+ A_t^\mathsf{T} + Q_t,$$

where $\mathcal{T}_t$ is the SSR/spot-vol transport of Note 12 and $A_t = \partial \mathcal{T}_t / \partial x$ is its local Jacobian. For a small v1, $A_t = I$ is acceptable for ATM/skew/curvature if the state is recomputed after transport; the important point is that process covariance Q_t grows when the prediction is less trustworthy.

Useful process-noise components are

$$Q_t = Q_{\text{clock}}(\Delta t) + Q_{\text{spot}}(|h|) + Q_{\text{event}} + Q_{\text{source}} + Q_{\text{model}},$$

with the following meanings:

Clock noise.
Volatility surfaces diffuse through time. A square-root-time scale, $q_h^2 \Delta t$ per handle, is the natural baseline.

Spot transport noise.
The larger the log-forward move $h = \log(F_t/F_{t-1})$, the less certain the transported state. This is the same intuition as the transport-distance factor in prior persistence.

Event noise.
Known event windows should widen the prediction before the event is resolved. The event clock changes the time variable; this term admits that the shape response itself is uncertain.

Source noise.
Switching provider, as-of mode, or live/previous-close source should widen or reset the filter. A live quote stream and a prior-close snapshot are not the same stochastic clock.

Model noise.
If the display model changes from LQD to SVI or Multi-Core SIV, the filtered handle state may persist, but model-specific parameters should not.

> **Caution**
> Do not filter native local-vol parameters. The local-vol grid is a projection machinery with its own PDE stability and no-arbitrage constraints. Filter the target smile handles, then project or refit the local-vol surface to that target.

6 The clean architecture

There are two mathematically safe ways to make the filter active.

6.1 Overlay mode: compute, display, do not affect calibration

Overlay mode is the recommended first delivery. The workflow is:

1. Fetch or stream an option snapshot.
2. Prepare quotes through the existing pipeline.
3. Run the current data-only calibration, with prior persistence off for the measurement pass.
4. Extract z_t and R_t.
5. Predict (m_t^-, P_t^-) from the previous filtered state.
6. Apply (4); store (m_t^+, P_t^+) and diagnostics.
7. Draw raw calibration, transported prediction and filtered handles/curve.

This mode cannot double-count quotes because it does not feed the posterior back into the fit. It is therefore ideal for tuning Q_t, R_t and reset rules.

6.2 Active mode: one-stage MAP, not posterior plus quotes

Active mode should not first build m_t^+ from today's quotes and then add m_t^+ as a prior while fitting the same quotes again. That counts q_t twice. The correct active objective is the one-step MAP problem

$$\mathcal{L}_{\text{active}}(\theta) = \mathcal{L}_{\text{quote}}(\theta; q_t) + \tfrac{1}{2}\Big\|\mathcal{H}(\theta) - m_t^-\Big\|^2_{(P_t^-)^{-1}} + \mathcal{L}_{\text{intrinsic}}(\theta) + \mathcal{L}_{\text{persist},\perp}(\theta). \tag{7}$$

Here $\mathcal{H}(\theta)$ extracts the filtered handle coordinates from the model. The second term is the Kalman prediction prior. The quote likelihood is today's market. In the linear-Gaussian case, minimizing (7) gives the same m_t^+ as (4). In the nonlinear smile case it is the extended Kalman/MAP version.

$\mathcal{L}_{\text{persist},\perp}$ is the remaining prior-persistence term, but only on coordinates not already covered by the filter's prediction prior or on regions outside the filtered handle state. This is the clean split:

$$\text{Kalman term} = \text{temporal prior on observed latent state},$$
$$\text{persistence term} = \text{gap prior on unobserved functionals}.$$

Proposition 2 (No double counting in the MAP form)**.** *If the model is linear, the quote extractor is $z = Hx + \epsilon$ with $\epsilon \sim \mathcal{N}(0, R)$, and the prediction is $x \sim \mathcal{N}(m^-, P^-)$, then the minimizer of*

$$\tfrac{1}{2}\|z - Hx\|^2_{R^{-1}} + \tfrac{1}{2}\|x - m^-\|^2_{(P^-)^{-1}}$$

is the Kalman posterior mean in (4).

Proof. The first-order condition is

$$(H^\mathsf{T} R^{-1} H + (P^-)^{-1})x = H^\mathsf{T} R^{-1} z + (P^-)^{-1} m^-.$$

The Woodbury identity rewrites the inverse of the left-hand side as $P^- - P^- H^\mathsf{T}(HP^-H^\mathsf{T} + R)^{-1} HP^-$. Substitution gives $x = m^- + P^- H^\mathsf{T}(HP^-H^\mathsf{T} + R)^{-1}(z - Hm^-)$, which is (4). □

6.3 Where prior persistence remains

Prior persistence should remain available in active mode, but with explicit semantics:

- It may act on deep tails or operator baskets not represented in the Kalman state.
- It may provide the initial saved prior from which the first filtered state is seeded.
- It may act on dark graph nodes that have no current quote observation.
- It should not also anchor ATM/skew/curvature to the same previous state that already appears as m_t^- in the Kalman term.

7 A telling case file

Case file: close-strike contradiction versus true gap

Setup. A one-month equity smile has nine near-ATM quotes and no reliable quotes beyond 25Δ. The previous filtered state transported to today is

$$m^- = (20.0\%,\ -0.35,\ 0.10), \qquad \sqrt{\operatorname{diag} P^-} = (0.30\%,\ 0.08,\ 0.05).$$

The near-ATM quotes imply a data-only handle observation

$$z = (20.4\%,\ -0.37,\ 0.55).$$

The ATM level and skew are consistent, but one stale close strike forces a very large curvature if the fit chases mids literally.

Diagnosis. Quote density alone would say the region is well observed. Therefore prior persistence should switch off for ATM/skew/curvature; using it to pull curvature back would violate the Note 13 rule. The contradiction should instead appear as measurement noise. Suppose the observation standard deviations after residual inflation are

$$\sqrt{\operatorname{diag} R} = (0.15\%,\ 0.05,\ 0.30).$$

The scalar gains are approximately

$$K_\sigma = \frac{0.30^2}{0.30^2 + 0.15^2} = 0.80, \quad K_s = \frac{0.08^2}{0.08^2 + 0.05^2} = 0.72, \quad K_\kappa = \frac{0.05^2}{0.05^2 + 0.30^2} = 0.027.$$

Filter. The update moves the real observed level and skew:

$$m_\sigma^+ = 20.0 + 0.80(20.4 - 20.0) = 20.32\%, \qquad m_s^+ = -0.35 + 0.72(-0.02) = -0.364.$$

But it barely accepts the isolated curvature kink:

$$m_\kappa^+ = 0.10 + 0.027(0.55 - 0.10) \approx 0.112.$$

Persistence. The unquoted wing is a different matter. For a deep-tail functional O_{tail}, observation precision is below requirement, so the activation gate of (1) is near one and the transported prior supplies the tail anchor. The same day therefore has both effects: Kalman filtering denoises the observed cluster, while prior persistence fills the wing gap.

This example is deliberately small, but it captures the implementation rule: a dense cluster with a bad internal residual is not a gap; it is a high-R_t observation. A strike range with no reliable quotes is a gap; it belongs to prior persistence.

8 Implementation plan

8.1 State objects

Add a pure numerical module, proposed as `backend/volfit/calib/observation_filter.py`, with immutable dataclasses:

`FilterState`.
Node key, as-of/source key, handle names, posterior mean m^+, covariance P^+, timestamp, provenance, and reset reason.

`FilterPrediction`.
Predicted mean m^-, covariance P^-, transport distance, process-noise breakdown.

`FilterMeasurement`.
Observed handles z, covariance R, quote-count/spread/RMS/inconsistency breakdown.

`FilterUpdate`.
Innovation $z - Hm^-$, innovation covariance S, gain K, posterior mean and covariance.

The numerical core should be pure numpy: no app state, no provider calls, no calibration side effects.

8.2 API and state integration

An app layer, proposed as `backend/volfit/api/observation_filter.py`, would:

1. Resolve the node key (ticker, expiry, fitMode, source, asOf).
2. Read prepared quotes and the latest data-only fit.
3. Build the measurement and covariance.
4. Read the previous filter state or seed from an active transported prior.
5. Apply the update and store diagnostics on `AppState`.
6. Expose a payload to the Smile and Graph views.

The state must reset on source/as-of changes, expiry changes, fit-mode changes that alter the observation loss, provider switches, large calendar gaps, and manual quote edits that invalidate the measurement.

8.3 Settings

The first surfaced settings should be deliberately few:

Table 1: Proposed observation-filter settings.

Knob	Default	Role
`observationFilterMode`	`off`	`off` / `overlay` / `active`.

Knob	Default	Role
`observationFilterState`	ATM / skew / curv	Handle coordinates carried by the filter.
`filterProcessVolBpSqrtDay`	5–15 bp	Clock process noise for ATM level; skew/curvature scale separately.
`filterTransportNoiseScale`	0.10	Extra process noise per log-forward transport distance.
`filterResidualInflation`	on	Use (6) to inflate R_t.
`filterMaxGain`	1.0	Optional safety cap; useful during pilot, removable later.
`filterResetHours`	session/EOD dependent	Maximum gap before the state is reset rather than predicted.

Hidden constants can mirror the graph precision scales: per-handle confidence, precision floors/-caps, spread half-life and quote-density saturation.

8.4 Workflow placement

The placement in the existing workflow is:

$$\text{fetch snapshot} \to \text{prepare quotes} \to \text{data-only fit}$$
$$\to \text{measurement } (z_t, R_t) \to \text{predict/update filter} \to \text{overlay or active MAP calibration.}$$

For a background calibration job, this belongs between the prepared-quote plan and the final fit commit. For live streaming, the filter may update every snapshot while active calibration remains trigger-gated; overlay mode is useful here because it shows whether the filter is tracking or lagging before it is allowed to steer calibration.

9 Validation protocol

The filter should not be accepted because charts look smooth. It needs a temporal backtest that distinguishes denoising from signal damping.

1. **Consecutive snapshots.** Use stored chain snapshots with real timestamps. Fit $t-1$, predict t, update on a thinned or noisy subset of t, and score held-out quotes at t.
2. **Close-strike contradiction injection.** Perturb one or two close strikes inside the spread model and verify that curvature noise is rejected without suppressing level moves.
3. **Shock pass-through.** Raise the whole ATM cluster by a true volatility jump with tight spreads. The filter must move with high gain.
4. **Gap preservation.** Remove the wings. The Kalman layer should not invent tail information; prior persistence or graph reconstruction should carry that load.
5. **Double-count guard.** Active mode must match the one-stage MAP objective. A test should fail if the implementation applies a posterior target and raw quotes as independent evidence.

6. **Calibration of uncertainty.** Standardized residuals, $(\text{heldout} - m^+)/\sqrt{P^+ + R_{\text{heldout}}}$, should have mean near zero and standard deviation near one, or conservatively below one.

Success is not "lower every RMS." Success is lower held-out error in noisy snapshots, lower shape jitter in repeated live refreshes, and no meaningful lag on high-quality market moves.

10 Existing anchors and proposed additions

Table 2: How the proposed filter should attach to today's code.

Responsibility	Existing anchor	Proposed addition
Raw quote containers and snapshot timestamps	`backend/volfit/data/types.py`	Use snapshot timestamp/source in the filter node key.
Quote preparation, de-Americanization and IV bands	`backend/volfit/api/quotes.py`	Build measurements only after prepared quotes exist.
Bid–ask objective and band semantics	`backend/volfit/calib/band.py`	Map inactive band rows into low measurement precision.
Calibration-consistent RMS	`backend/volfit/calib/rms.py`	Use fit residuals for R_t inflation.
Shared precision vocabulary	`backend/volfit/calib/precision.py`, `backend/volfit/graph/precision.py`	Reuse density, spread, freshness and caps for cheap R_t.
Prior-persistence builders and gates	`backend/volfit/calib/prior.py`, `backend/volfit/calib/operators.py`, `backend/volfit/calib/factors.py`	Keep gap priors separate from the Kalman prediction prior.
Covariance-form Gaussian update pattern	`backend/volfit/graph/posterior.py`	Mirror its marginal-variance and innovation diagnostics.
Workflow fetch/calibrate/stream placement	`backend/volfit/api/workflow.py`, `backend/volfit/api/service.py`	`backend/volfit/api/observation_filter.py`.
Numerical filter core	None today	`backend/volfit/calib/observation_filter.py`.
Backtest harness	`backend/backtest`, `backend/tests/test_temporal_backtest.py`	`backend/backtest/observation_filter.py`, `backend/tests/test_observation_filter.py`.

11 What is original here

The Kalman equations are classical [1, 2]. The Vol-Fitter-specific contribution is the separation of three notions that are usually blended in volatility tools:

1. **Quote likelihood**: today's prepared quotes, with bands, weights and no-arbitrage model structure.

2. **Temporal filtering**: a transported latent smile state with prediction and measurement covariance.

3. **Gap persistence**: a prior that activates only where current quotes do not identify a functional.

That separation is what makes the feature safe. The filter can denoise a noisy observed smile without using yesterday as a fake quote, and prior persistence can stabilize unobserved wings without damping a real current-market move.

A Hyperparameter atlas

Table 3: Observation-filter hyperparameters, proposed v1.

Knob	Default	Role
Surfaced		
`observationFilterMode`	`off`	Feature mode: `off`, `overlay`, `active`.
`observationFilterState`	ATM / skew / curv	Handle set carried by the filter.
`filterProcessVolBpSqrtDay`	10 bp	ATM process noise per square-root calendar day.
`filterProcessSkewSqrtDay`	0.02	Skew process noise scale.
`filterProcessCurvSqrtDay`	0.05	Curvature process noise scale.
`filterTransportNoiseScale`	0.10	Process-noise multiplier for large spot/forward transports.
`filterResetHours`	source dependent	Reset rather than predict across long data gaps.
`filterMaxGain`	1.0	Pilot safety cap on diagonalized gains; should not bind in normal operation.
Hidden / derived		
`HANDLE_CONFIDENCE`	$[1, 1, 0.01]$ initially	Curvature is less identified than level and skew, matching graph precision practice.
`RMS_FLOOR`	existing precision floor	Prevents one excellent fit from creating infinite observation precision.
`SPREAD_HALF`	existing precision half-life	Wider bid–ask bands inflate R_t.
`REF_ATM_QUOTES`	existing density saturation	Dense ATM support raises level/skew precision.
`RESID_INFLATION_CAP`	TBD	Caps (6) during the pilot to avoid one broken chain poisoning the state.
`SOURCE_RESET_POLICY`	strict	Reset on provider/as-of/source changes unless an explicit bridge is implemented.

B Performance notes

1. **Overlay mode is cheap after calibration.** If a data-only fit has already been computed, the Kalman update is a dense $d \times d$ solve with $d \leq 6$ in v1. The cost is invisible next to calibration.

2. **The expensive part is covariance extraction.** Full Jacobian-propagated R_t requires either existing analytic Jacobians or a Gauss–Newton approximation from the optimizer. The cheap

precision builder (4.3) is a sensible first overlay.

3. **Active MAP should not add a second fit pass by default.** The prediction prior is just another residual block in the existing least-squares stack. A diagnostic data-only prepass can improve R_t, but it should be an explicit setting just like Note 13's prior prepass.

4. **State storage is small.** Per node, the state is $O(d^2)$: a mean, covariance and a few diagnostics. Even thousands of nodes are negligible.

5. **Graph generalization is natural but later.** A graph Kalman filter would replace the per-node P_t^- with a block covariance over the universe. That is Note 14's covariance-form posterior with time dynamics added; it should come after the single-node filter is validated.

C Reference implementation

The numerical heart is small. The listing below is intentionally model-free: it assumes the caller has already built the prediction and measurement moments.

Listing 1: Covariance-form Kalman update with Joseph covariance.

```
def kalman_update(mean_pred, cov_pred, obs, obs_cov, H=None):
    m = np.asarray(mean_pred, dtype=float)
    P = np.asarray(cov_pred, dtype=float)
    z = np.asarray(obs, dtype=float)
    R = np.asarray(obs_cov, dtype=float)
    H = np.eye(m.size) if H is None else np.asarray(H, dtype=float)

    innovation = z - H @ m
    S = H @ P @ H.T + R
    K = np.linalg.solve(S.T, (P @ H.T).T).T

    out_mean = m + K @ innovation
    I_KH = np.eye(m.size) - K @ H
    out_cov = I_KH @ P @ I_KH.T + K @ R @ K.T
    out_cov = 0.5 * (out_cov + out_cov.T)
    if np.any(np.linalg.eigvalsh(out_cov) < -1e-12):
        raise FloatingPointError("posterior covariance lost PSD")
    return out_mean, out_cov, innovation, S, K
```

The active calibration residual corresponding to (7) is also small once the model exposes a handle extractor:

Listing 2: Residual rows for the active MAP prediction prior.

```
def prediction_prior_residual(model_handles, mean_pred, cov_pred):
    L = np.linalg.cholesky(np.asarray(cov_pred, dtype=float))
    return np.linalg.solve(L, np.asarray(model_handles) - np.asarray(
        mean_pred))
```

In production the Cholesky solve should use a small jitter and report it; adding jitter is a diagnostic event, not a silent repair.

References

[1] R. E. Kalman. A new approach to linear filtering and prediction problems. *Journal of Basic Engineering*, 82(1):35–45, 1960.

[2] B. D. O. Anderson and J. B. Moore. *Optimal Filtering*. Prentice-Hall, 1979.

[3] A. Gelman et al. *Bayesian Data Analysis*. CRC Press, 3rd ed., 2013.

[4] J. Gatheral. *The Volatility Surface*. Wiley, 2006.